

大模型生码的原理与 RAG 工程实践

TL;DR (摘要)

1. 大模型生码：兴奋之后，为什么总是失望？
2. 大模型到底是怎么“写代码”的？
3. 为什么代码“看起来对”，却经常是错的？
4. LLM 生码的本质：不是推导，而是延续模式
 - 4.1 上下文里已经出现过什么写法
 - 4.2 token 之间的共现关系
 - 4.3 写代码时的惯性
5. 为什么“信息给得更多”，反而更不稳定？
 - 5.1 方案一：组件文档（API 文档）
 - 5.2 方案二：组件完整 TS 类型
 - 5.3 方案三：组件完整源码
6. 对大模型最友好的输入是什么？——代码示例
7. 原子化 RAG：真正的作用是“铺轨道”
8. 当前原子化 RAG 的优势与劣势
 - 8.1 优势：它把“生成问题”变成了“工程问题”
 - 8.2 劣势：它不是轻方案，也不是一劳永逸
9. 那为什么现在的 Agent 看起来“能理解业务规则”？
10. Agent 本质上在做什么？
 - 10.1 Agent 和原子化 RAG，其实是一条思路的两个层级
 - 10.2 一个重要但容易误解的点
 - 10.3 把这两件事合在一起看，会得到一个非常清晰的结论
11. 再往前一步：Skill + 本地文件系统，会不会取代原子化 RAG？
 - 11.1 先分清两层：取数方式 vs 决策方式
 - 11.2 Skill 真正改变的是什么？
 - 11.3 把重复的工程动作固化下来
 - 11.4 让原子化 RAG 更容易规模化
 - 11.5 更容易接入真实反馈闭环

11.6 那什么不会变？

11.7 一个很现实的风险：skill 会诱发“全量塞”的幻觉

11.8 把未来的形态说清楚

12. 收尾

TL;DR (摘要)

- 大模型写代码并不是“理解业务后再实现”，而是在 **预测下一个 token**
- 生码不稳定的根因，不是 prompt 不够强，而是 **模型在猜**
- 文档、TS、源码信息更多，但并不更稳，反而容易稀释关键信号
- 代码示例是对 LLM 最友好的输入形态，因为它们是已验证的高概率模式
- 原子化 RAG 的核心价值，是把“生成问题”变成“工程问题”
- Agent 看起来能理解规则，并不是模型变聪明，而是系统把生成拆解、约束、反馈了
- Skill + 本地文件系统 不会取代原子化 RAG，只会让这套工程思路更可执行、更规模化
- 这是一条长期可演进的工程路线，而不是短期技巧

1. 大模型生码：兴奋之后，为什么总是失望？

现在越来越多团队开始用大模型来写代码。

一开始大家都很兴奋：

“哇，它居然能直接写 React / SQL / 后端接口。”

但只要真的在项目里用过一段时间，基本都会遇到同一个问题：

代码看起来很对，但就是不太能用。

有时候能跑，有时候不行。

稍微改一点需求，生成结果就开始飘。

于是大家开始疯狂调 prompt、加规则、写长说明，

甚至寄希望于：“等模型再聪明一点就好了。”

但说实话，这条路本身就有点走歪了。

因为问题不在于你“怎么用”，

而在于我们对大模型到底在干嘛这件事，本身就理解错了。

2. 大模型到底是怎么“写代码”的？

现在的大语言模型，不管是 GPT、Claude 还是 LLaMA，本质上都可以理解成一类东西：

基于 Transformer 架构训练出来的自回归语言模型。

这句话听起来有点技术，但意思其实很简单。

翻译成人话就是：

它不会“思考怎么写代码”，它只是在做一件事：

根据已经出现的内容，猜下一个最有可能出现的 token 是什么。

你让它写代码也好，写文章也好，对它来说流程是完全一样的：

```
看到前面的 token  
→ 预测下一个 token  
→ 再下一个  
→ 一直往下接
```

并不存在什么：

- 先理解业务
- 再设计结构
- 最后实现代码

3. 为什么代码“看起来对”，却经常是错的？

因为从模型视角看，代码只是一种比较规整的文本。

比如下面这段：

```
1  <ProTable  
2    columns={columns}  
3    dataSource={data}  
4    pagination={true}  
5    editable={true}  
6  />
```

TSX

站在人类工程师的角度，你一眼就能看出问题：

- `editable` 很可能不是 `boolean`
- `pagination={true}` 在真实组件里可能根本不支持

但对模型来说，这段代码的问题只有一个：

它在统计意义上“看起来像是常见写法”。

模型并不知道：

- 这个 `prop` 在真实运行时是否合法
- 这个组件有没有这样的组合方式
- 你们团队到底推荐怎么用

它只是觉得：

“在 React 表格代码附近，这样写的概率挺高。”

于是它就写了。

4. LLM 生码的本质：不是推导，而是延续模式

这是一个非常关键、但经常被忽略的点。

大模型在生码时，真正依赖的其实只有三件事。

4.1 上下文里已经出现过什么写法

如果你在上下文里给过它这样的代码：

```
▼ TSX |
1  <ProTable
2    rowSelection={{ type: 'checkbox' }}
3    pagination={{ pageSize: 20 }}
4  />
```

那它后面再写 `ProTable`，就会非常自然地继续用 `object` 形式写 `pagination`。

不是因为它“理解了 API”，而是因为 `token` 序列已经被证明是高概率、合理的。

4.2 token 之间的共现关系

模型学到的是类似这种经验：

- `rowSelection` 后面大概率跟一个 `object`
- `onChange` 后面经常是一个函数
- `useState` 往往和数组解构一起出现

这些不是规则，是统计关系。

4.3 写代码时的惯性

一旦模型进入某种写法轨道，比如 `hooks` 风格、受控组件写法，它很少会主动跳出去。

可以把它理解成：

一旦上了轨道，就会顺着轨道往前滑。

5. 为什么“信息给得更多”，反而更不稳定？

很多团队在这里会自然地尝试几条路：

- 把组件 API 文档塞进 `prompt`
- 把组件完整 TS 类型塞进去
- 甚至把组件完整源码都喂给模型

这些做法看起来都很合理，但实践中稳定性往往并不好。

5.1 方案一：组件文档（API 文档）

▼

TSX

```
1  editable?: { type: 'single' | 'multiple' }
```

这是给人看的。

模型需要先把自然语言理解成结构，再决定怎么写成代码，中间全是自由发挥空间。

5.2 方案二：组件完整 TS 类型

TS 对人和编译器是强约束，但对 LLM 来说，只是一段文本。

除非你有完整的“生成 → 编译 → 报错 → 再修”闭环，否则 TS 本身并不能保证稳定。

5.3 方案三：组件完整源码

源码回答的是：

组件是怎么实现的

而生码真正需要的是：

组件在真实工程里通常是怎么用的

源码里的实现细节，很容易误导模型选中“存在但不该用”的路径。

6. 对大模型最友好的输入是什么？——代码示例

▼

TSX |

```
1  <ProTable
2    editable={{ type: 'multiple', onSave }}
3    rowSelection={{ type: 'checkbox' }}
4  />
```

对模型来说，这是几乎完美的输入：

- token 都是已验证过的
- 组合关系稳定
- 模式在训练中反复出现

模型不需要理解“为什么”，只需要 复制、微调、拼装。

7. 原子化 RAG：真正的作用是“铺轨道”

原子化 RAG 并不是“多找资料”，
而是：

- 裁剪可能性
- 显式化示例
- 限制自由生成

把模型从“乱猜”，放到“组合已知模式”的轨道上。

8. 当前原子化 RAG 的优势与劣势

如果把原子化 RAG 的“示例驱动生码”当成一套工程方案来看，它的 trade-off 非常清晰。

8.1 优势：它把“生成问题”变成了“工程问题”

- 输入是可控、可版本化的
- 生成路径是可追溯的
- 出错时可以明确定位在哪一层

你可以清楚回答：

- 用了哪个组件
- 参考了哪个示例
- 是需求解析错了，还是示例不全

这已经不是“AI 魔法”，而是一套工程流水线。

8.2 劣势：它不是轻方案，也不是一劳永逸

- 前期建设成本明显更高
- 对物料质量高度敏感
- 扩展新场景需要持续补物料
- 对系统设计能力要求高

它不适合追求“一句 prompt 出奇迹”，但非常适合追求长期稳定。

9. 那为什么现在的 Agent 看起来“能理解业务规则”？

在解释原子化 RAG 之后，很多人会自然产生一个疑问：

既然大模型本质还是在猜 token，那为什么现在的 Agent 看起来却能理解业务规则、做多步决策、还能自我修正？

答案其实并不复杂。

Agent 并没有让模型突然“懂业务”，而是把一次不稳定的生成，拆解成了一连串可控的生成过程。

10. Agent 本质上在做什么？

从工程角度看，Agent 做的事情可以总结成三点：

- 1. 把一个大问题，拆成很多小问题
- 2. 把中间状态显式写出来，重新喂回模型
- 3. 用外部规则和反馈，不断剪掉错误路径

模型本身，仍然在做那件事：预测下一个最有可能的 token。

只是这一次，它不是在一个“大而模糊”的上下文里猜，而是在很多个小而清晰的上下文里，反复猜。

这就制造了一种强烈的“它在思考”的感觉。

10.1 Agent 和原子化 RAG，其实是一条思路的两个层级

如果把两者并排放在一起看，你会发现它们解决的是同一类问题：

如何减少模型需要“自由猜测”的空间。

下面这张对照表，基本可以把两者的关系说清楚。

原子化 RAG 在做什么	Agent 系统里发生的事情
拆解组件能力	拆解复杂任务为多步

提供显式代码示例	提供显式中间状态 (plan / thought / step)
裁剪可选组件与模式	剪枝失败路径、回避错误决策
固定高概率生成轨道	在小上下文中反复约束生成
可追溯生成路径	可回放 reasoning 与执行过程

可以看到：

- 原子化 RAG 更偏向“输入侧工程”
- Agent 更偏向“过程侧工程”

但他们的核心目标是一致的：把生成问题，从一次不可控的猜测，变成一连串可控的小选择。

10.2 一个重要但容易误解的点

Agent 的 thinking，并不是模型内部多了一套“推理引擎”。

它更像是：

被拆解、被记录、被反馈约束的 token 生成过程。

- thinking 是写出来的
- reasoning 是可回放的
- 修正是靠外部反馈完成的

这和“示例驱动生码”的思路是完全一致的，只是粒度更细、步骤更多。

10.3 把这两件事合在一起看，会得到一个非常清晰的结论

- 原子化 RAG 解决的是：“在给定能力空间里，怎么稳定生成”
- Agent 解决的是：“在多步决策中，怎么持续不跑偏”

它们不是对立方案，而是同一工程哲学在不同层级的体现。

Agent 看起来能理解业务规则，并不是因为模型本身发生了质变，而是因为系统用拆解、结构和反馈，把模型的生成路径一步步收紧。

从原子化 RAG 到 Agent，本质上都是在做同一件事：减少猜测，增加约束，把生成问题变成工程问题。

11. 再往前一步：Skill + 本地文件系统，会不会取代原子化 RAG？

在讲清楚原子化 RAG 和 Agent 之后，很多人会继续问一个更长期的问题：

如果未来用 skill 直接对接本地文件系统做 RAG，那现在这套原子化 RAG 的思路，会不会被抛弃？

不会，skill 和本地文件系统，只会改变“东西从哪来、怎么拿”，但不会改变“哪些东西该被拿进来、该怎么用”。

11.1 先分清两层：取数方式 vs 决策方式

这里有一个非常关键的区分，很多讨论之所以拧巴，都是把这两层混在了一起。

- **取数层 (Retrieval / Plumbing)**
 - 从哪里拿资料
 - 怎么索引、搜索、过滤
 - 是向量库、BM25，还是本地文件系统 keyword + grep ?
 - 是一次性拉，还是按需拉
- **决策层 (原子化 RAG 的核心)**
 - 需求如何结构化
 - 能力如何路由
 - 示例如何作为主轨道
 - 上下文如何裁剪
 - 生成路径如何可追溯

现在做的原子化 RAG，价值几乎全部在决策层。而 skill，主要强化的是取数层和执行层。
所以它们不是替代关系，而是叠加关系。

11.2 Skill 真正改变的是什么？

如果把 skill 引入进来，变化会非常明显，但方向是“变稳”，不是“变玄”。

11.3 把重复的工程动作固化下来

很多你现在在 pipeline 里手写或硬编码的步骤，其实都非常适合变成 skill：

- 从本地代码库中检索某个组件的示例
- 从 TS 中抽取 props 约束
- 按 token budget 裁剪上下文
- 运行 tsc / test，收集错误
- 根据错误信息补充缺失示例或约束

skill 的价值在于：把“工程动作”变成可复用、可组合的执行单元。

11.4 让原子化 RAG 更容易规模化

当你的 pipeline 变成一组 skill 之后：

- 不同团队可以复用同一套能力
- 不同场景只需要换配置，而不是重写逻辑
- 决策路径天然可记录、可回放

这一步，其实是把原子化 RAG 从“方法论”，推进到“平台能力”。

11.5 更容易接入真实反馈闭环

skill 很自然就能接入：

- 编译器
- 单元测试

- 运行时校验
- lint / 规则引擎

这意味着：

生成不再是“一次性事件”，而是一个可以被反馈不断修正的过程。

但注意：这一步是增强稳定性，不是让模型“更聪明”。

11.6 那什么不会变？

哪怕未来你完全基于本地文件系统 + skill 做 RAG，有几件事依然不会变：

- 示例依然是主轨道
- TS / 源码依然只是“补洞和校验”
- 上下文依然需要裁剪，而不是全量塞
- 决策路径依然需要被显式记录

如果你放弃这些原则，skill 只会让你更快地把错误规模化。

11.7 一个很现实的风险：skill 会诱发“全量塞”的幻觉

skill + 本地文件系统，非常容易让人产生一种错觉：

“既然拿文件这么方便，不如把整个 repo 都给模型。”

这一步，恰恰是回到不稳定区间的开始：

- 注意力被稀释
- 推荐用法被实现细节淹没
- 内部 hack 被误当成最佳实践

所以原则反而要更严格：检索越方便，越要坚持原子化。

11.8 把未来的形态说清楚

如果用一句话总结这套体系未来最自然的演进形态：

原子化 RAG 是架构，skill 是执行器，本地文件系统是物料面。

- 架构决定稳定性上限
- 执行器决定工程效率
- 物料面决定覆盖范围

三者是正交增强，而不是彼此替代。

12. 收尾

从原子化 RAG，到 Agent，再到 skill，本质上没有发生路线切换。

发生的只是同一件事在不同层级上的展开：

不断减少模型需要“猜”的地方，把生成问题，彻底工程化。

当用这个视角回看，会发现：

- Agent 并没有否定示例驱动
- skill 也没有否定原子化 RAG
- 它们都只是让这套思路更可执行、更可规模化

这条路不是短期技巧，而是一条长期可演进的工程路线。